Frameworks

Un framework se puede ver como el resultado de refactorizar una aplicación, no solo para mejorar su calidad interna sino para abstraer lo que es común todas una familia de aplicaciones similares. En ese proceso de refactoring se identifica lo que se mantendrá constante y será, a partir de ese momento, responsabilidad de los desarrolladores del framework y lo que podrá variar de aplicación a aplicación y será responsabilidad de los desarrolladores de aplicaciones que usen el framework.

Diferencia entre frameworks y librerias:

Libreria: ***conjunto de "clases útiles"***

- **Resuelven problemas comunes a la mayoría de aplicaciones**
- **Cada clase de la librería resuelve un problema concreto, es independiente del contexto del uso y generalmente también es independiente de otras cases en la librería**
- **Nuestro código controla/usa a los objetos de las librerías**

Framework: ***estructura coherente que define un esqueleto "semi-completo"***

- **Aplicación semi-completa y reusable que puede ser especializada para producir aplicaciones a medida**
- **Conjunto de clases concretas y abstractas, relacionadas (por herencia, composición, envio de mensajes) de manera que resuelven la mayor parte del problema en cuestión (Conforman un todo)**
- **Los desarrolladores incorporan al framework en sus aplicaciones y lo especializan para cubrir sus necesidades.**

IMPORTANTE: En una librería se reusa código mientras que en un framework se reusa una manera de instanciar, de ejecutar y de extender

Inversión de control

- Permite que el flujo de control del programa sea invertido. En lugar que de que el código de la aplicación controle el flujo de la ejecución, un framework o contenedor externo toma el control y llama a los componentes según sea necesario

- Permite al framework determinar qué set de métodos específicos de la aplicación invocar en respuesta a eventos externos
- Entonces
    - Codigo del framework envia mensajes a mis clases
    - El framework ejecuta mis métodos e instancia mis clases (cosa que normalmente no sucedería)
    - CODIGO DEL FRAMEWORK INVOCA A MI CÓDIGO (El código del framework controla al nuestro)

Conclusión: La inversión de control es un mecanismo que poseen los frameworks que les permiten ejecutar código hecho por el desarrollador que lo implementa, ante determinados eventos, en lugar que el código de dicho desarrollador invoque al código del framework

Para separar la implementación de la parte constante (a lo que llamamos **frozenspot**) y de las partes que varían (a las que llamamos **hotspots**), se introducen en el framework puntos de extensión. Estos tienen la forma de plantillas que ofrecen ganchos para que, mediante herencia o composición, los programadores de aplicaciones definan el comportamiento variable. Algunos patrones de diseño (particularmente el template method y el strategy) son frecuentemente utilizados para introducir esos puntos de extensión.

La construcción y uso de frameworks es una estrategia para modularizar y reutilizar, que mejora la calidad del software y aumenta la productividad de quienes lo hacen. Un framework, visto como módulo de software, no solo encapsula operaciones y estructuras de datos reusables sino que las combina para resolver, con demostrada eficacia, los requerimientos comunes a una familia de aplicaciones. En pocas palabras, encapsula el diseño y la implementación de la parte más compleja de dichas aplicaciones y permite su reutilización.

Un framework es en sí mismo un módulo de software reutilizable. Muchas aplicaciones se construirán sobre él. El framework será independiente de las aplicaciones. Sin embargo, las aplicaciones dependerán de que la interface (conjunto de puntos de extensión) y comportamiento del framework se mantenga estable. Esto es importante dado que, a lo largo de su vida, el framework será foco de contínuo refactoring y de mejoras para eliminar bugs y hacerlo más efectivo.

FrozenSpot

- Elementos del framework que no se pueden modificar (marca indeleble)
- Conjunto de características que comparten todas las aplicaciones creadas por el mismo framework

HotSpot

- Puntos de extensión que ofrece el framework que nos permiten introducir variantes y así construir aplicaciones diferentes
- Los puntos de extensión pueden implementarse en base a herencia o en base a composición

Plantillas y ganchos

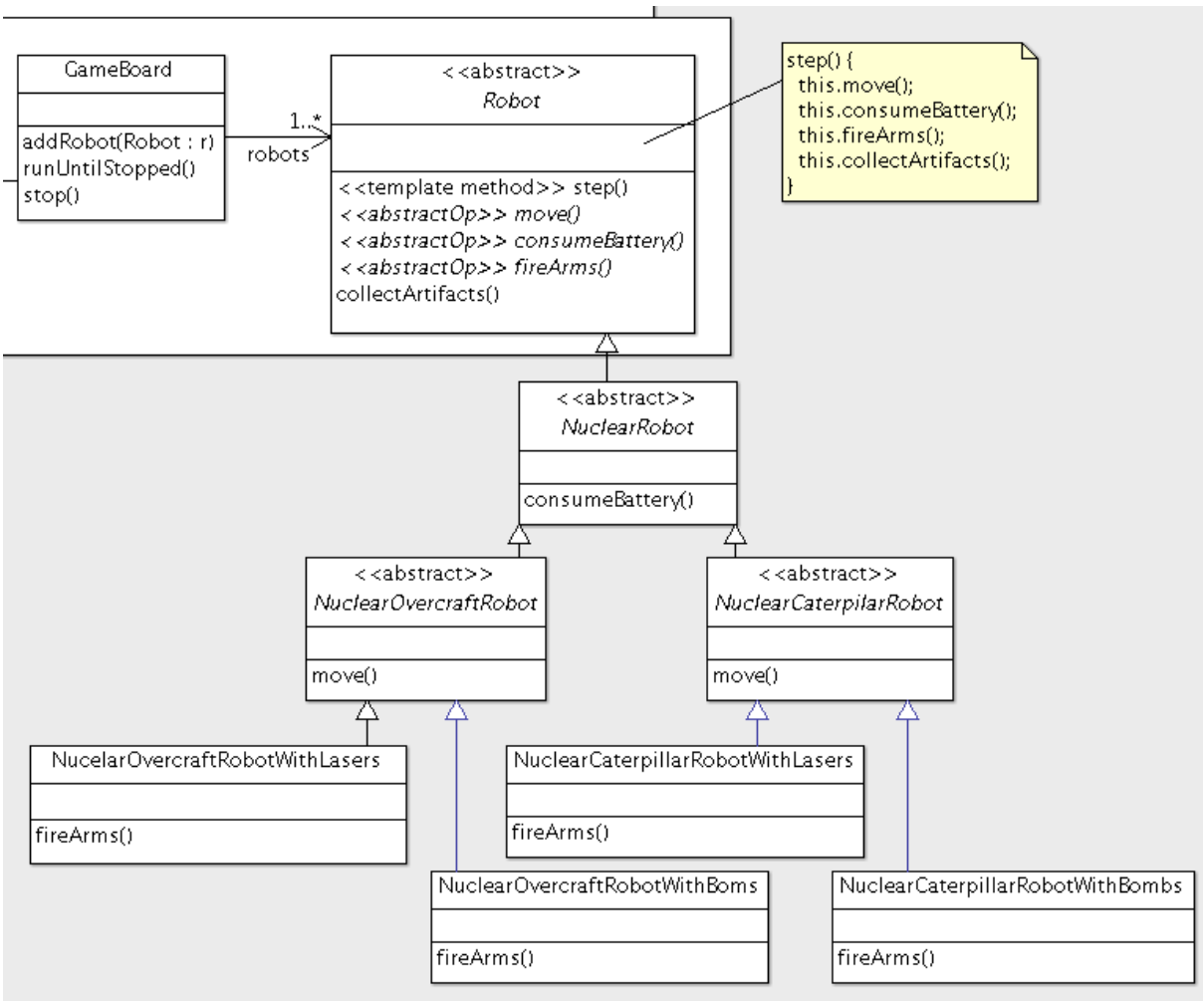
Las plantillas y los ganchos son una estrategia de programación comúnmente utilizada para introducir puntos de variabilidad (hotspots) en los frameworks orientados a objetos. Pueden implementarse utilizando herencia o composición como mecanismos para separar los aspectos comunes de los aspectos variables.

Plantillas y herencia:

El método plantilla es parte del frozenspot mientras que las operaciones abstractas (o los ganchos) dan lugar a la implementación de los hotspots.

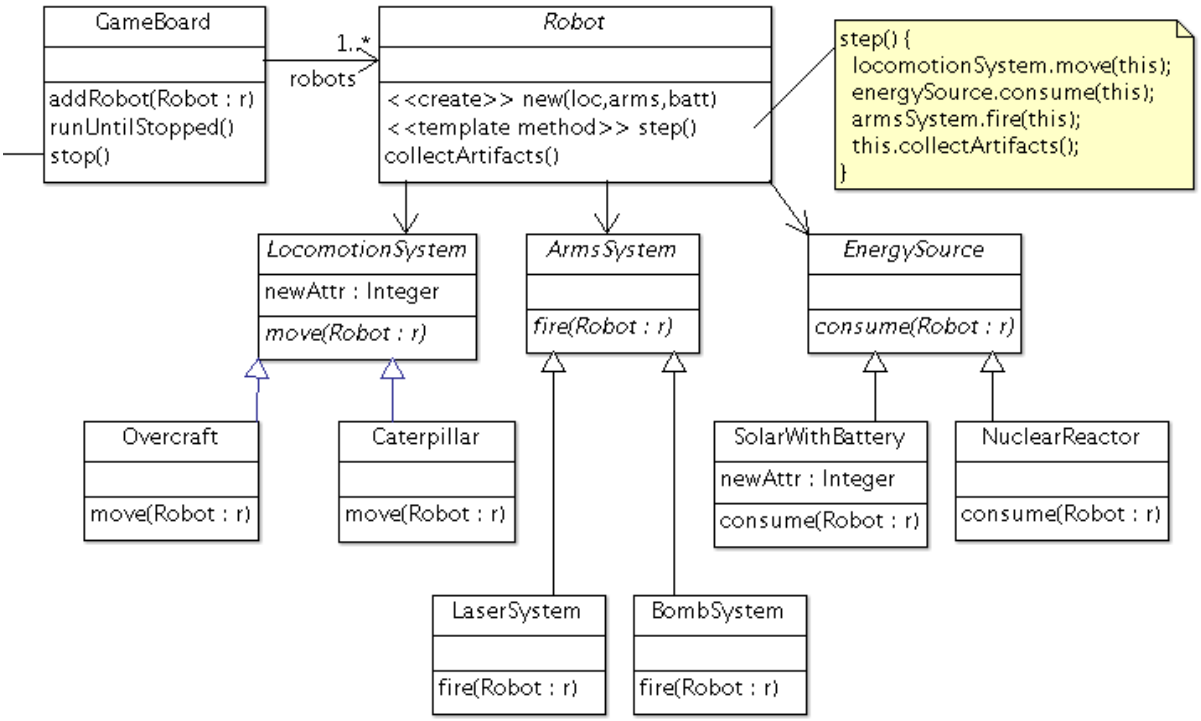
La explosión en el número de subclases y la duplicación de código evidencian las limitaciones de la herencia como estrategia para separar plantillas y ganchos. En situaciones como estas puede explorarse la composición como una estrategia alternativa.

Si uso herencia, la plantilla se implementa en una clase abstracta y los ganchos en sus subclases .



<u>Plantillas y composición</u>: No solo lo separa en métodos diferentes, sino que también lo encapsula en objetos diferentes

Importante:
Ahora, cuando un robot recibe el mensaje *step(),* sigue utilizando su método plantilla (que garantiza el cumplimiento de los pasos canónicos como lo marca la especificación del framework). Sin embargo el método plantilla ya no envía mensajes al objeto mismo (como lo hacía en el caso de herencia) sino que delega las operaciones en sus partes. Entonces, la plantilla está en el robot, y los ganchos en las partes.



Habrá notado que cuando el robot delega las operaciones gancho en sus partes, se pasa el mismo como parámetro. Esto es necesario porque las partes ya no tienen acceso a las variables de instancia del robot ni de las otras partes. En consecuencia al implementar los métodos ganchos (en las partes) solo se puede utilizar los métodos que el robot haga públicos. Esto resulta en un escenario mas restrictivo que el que utilizaba herencia.

Si uso composición, un objeto implementa la plantilla y delega la implementación de los ganchos en sus partes.

Resumen de lo que vimos:

- Las plantillas y los ganchos se pueden utilizar en un framework para separar lo que se mantiene constante (frozenspot) de lo que varía (hotspots).
- Las plantillas implementan lo que se mantiene constante, los ganchos lo que varía.
- Se puede utilizar herencia o composición para separar la plantilla de los ganchos.
- Si uso herencia, la plantilla se implementa en una clase abstracta y los ganchos en sus subclases (como el clásico patron Método Plantilla). Si uso composición, un objeto implementa la plantilla y delega la implementación de los ganchos en sus partes.
- Con herencia:

    - es mas simple para casos con pocas alternativas y/o pocas combinaciones
    - al implementar los métodos gancho puedo utilizar las variables de instancia y todo el comportamiento heredado de la clase abstracta
    - si hay muchas variantes o combinaciones, empiezo a tener muchas clases y duplicación de código
- Con composición:
    - evita la duplicación de código y el creciente numero de clases cuando existen muchas alternativas y combinaciones posibles
    - al implementar los métodos gancho debo pasar como parámetro todo lo que necesiten - no tienen acceso a las variables de instancia del objeto
    - puedo cambiar el comportamiento en tiempo de ejecución sin mayor dificultad

Canonicos

Frameworks de caja blanca

- Utilizan herencia en sus puntos de extensión
- Debemos conocer el framework para poder usarlo(como se concetan las piezas, como interactuan.
- Loop de control suele ser Template method

Frameworks de caja negra

- Utilizan composición en sus puntos de extensión
- No hace falta conocer el funcionamiento grandes detalles internos del framework
- Loop de control suele ser manejado por un contenedor o configurador

Por qué se dice que en las frameworks white box no hay encapsulamiento? Porque EchoServer hereda el loop de control no existe encapsulamiento ⇒ es una Caja Blanca

● La instanciación hereda y completa el loop de control ● Es posible que requiera agregar métodos a clases del framework ● Demanda conocimiento del código del framework

CONVERTIR FROZENSPOTS EN HOTSPOTS

```
72                  if (inputLine.equalsIgnoreCase("")) {
73                      break; // Client requested to close the connection
74                  }
```

Ejercicio: como podría convertir ese FrozenSpot en un HotSpot?

1. Extraigo la logica en un método configurable (hook method):

```
protected boolean connectionClosure(String inputLine) {
    return inputLine.equalsIgnoreCase("");
}
```

2. Reemplazo por la llamada al método hook en el loop de conexión:

```
if (this.connectionClosure(inputLine)) {
    break;
}
```

3. En la clase EchoServer (instanciación)

```
@Override
protected boolean connectionClosure(String inputLine) {
```

```
        return inputLine.equalsIgnoreCase("xd");
    }
```

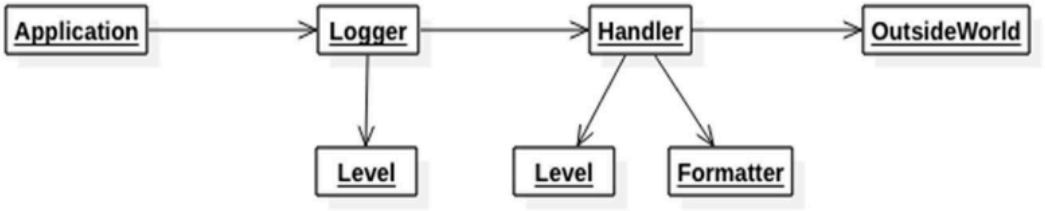
Para convertir ese FrozenSpot en un HotSpot, hay que extraer la lógica fija (como inputLine.equals("")) en un método que pueda ser sobrescrito por las instanciaciones del framework.

JAVA UTIL LOGGIN:

- Agregamos código de loggin a nuestra aplicación para entender lo que pasa con ella
- Los logs son útiles para desarrolladores, administradores y usuarios

Arquitectura visible

- Logger: objeto al que le pedímos que emita un mensaje de log
- Handler: encargado de enviar el mensaje a donde corresponda
- Level: indica la importancia de un mensaje y es lo que mira un
- Logger y un Handler para ver si le interesa
- Formater: determina cómo se ”presentará” el mensaje

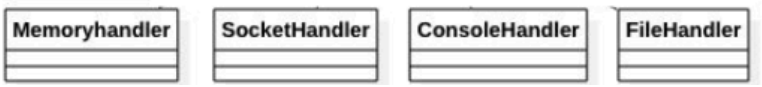



```
//Obtener instancias del logger y almacenarlo en una variable:
Logger.getLogger("app");
appLog= Logger.getLogger("app");
//Setear el nivel del logger:
Logger.getLogger("app").setLevel(Level.OFF);
appLog.setLevel(Level.SEVERE);
Logger customersLogger = Logger.getLogger("ecommerce.customers");

//Establece que el nivel mínimo de mensajes que este logger aceptará es
```

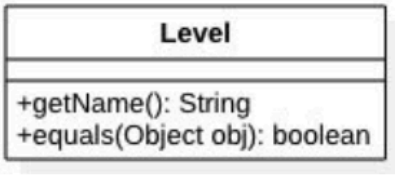
```
WARNING (solo logueará mensajes de nivel WARNING, SEVERE, o superior)
customersLogger.setLevel(Level.WARNING);

/* Crea un manejador de logs que guarda los mensajes en un archivo llamado
ecommerce-customers.log
Un Handler es lo que se encarga de dónde se envía el log (consola, archivo,
etc) */
FileHandler customersLoggerhandler= new FileHandler
("ecommerce-customers.log");

//Define como se formatea el mensaje de log
customersLoggerHandler.setFormatter(new SimpleFormatter());

/* Asocia el FileHandler al logger (le indica que escriba los mensajes al
archivo)
   Un logger puede tener multiples Handlers al mismo tiempo */
customersLogger.addHandler(customersLoggerHandler);
```

Handler

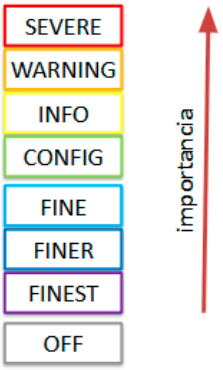
- Recibe los mensajes del Logger y determina como "exportarlos"
- Instancias de MemoryHandler, ConsoleHandler, FileHandler, o SocketHandler
- Puede filtrar por nivel
- Tiene un Formatter



Level

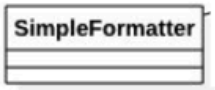


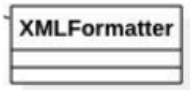
- Representa la importancia de un mensaje
- Cada vez que pido que se loggee algo, debo indicar un nivel
- Los Loggers y Handler comparan el nivel de un cada mensaje con el suyo para decidir si les interesa o no
- Si te interesa un nivel, también te interesan los que son más importantes que ese
- Hay niveles predefinidos, en variables estáticas de la clase Level (p.e., Level.OFF)



Formatter

- El Formatter recibe un mesaje de log (un objeto) y lo transforma a texto
- Son instancias de: SimpleFormatter o XMLFormatter
- Cada handler tiene su formatter
  - Los FileHandler tienen un XMLFormatter por defecto
  - Los ConsoleHandler tienen un SimpleFormatter por defecto





Parcial Frameworks 1er Recuperatorio 2024

1-Cada aplicación cuenta con un logger que posee un nivel que compra con el nivel de mensaje que recibe para aceptarlo si es igual o mas relevante que el suyo o rechazarlo si es menor. A su vez se cuenta con un handler que es el encargado de enviar el mensaje a donde corresponda. Este cuenta a su vez con un nivel y con un formatter que determina cómo se presentará el mensaje.

Las instancias del handler son consolehandler, memoryhandler, filehandler y storagehandler. Por defecto el consolehandler tiene el simpleformatter y el filehandler tiene xmlformatter.

Para usar un logger instancio un logger y le pido el logger con el nombre que le pase por parametro, si no existe lo crea.Entonces para hacer un logger que loguee en consola:

```
//creo una instancia de logger
Logger logger=Logger.getLogger("app")
//lo seteo con un nivel que quiera
logger.setLevel(Level.SEVERE)
```

como por defecto el handler es consolehandler, al llamar al logger y pasarle el mensaje, lo imprime en pantalla

```
logger.log(Level.SEVERE,"HOLA")
```

Para loguear en un archivo con formato xml le paso al handler un nuevo formatter que va a ser el xml

```
Logger loggerXML=Logger.getLogger("app.xml")
FileHandler file=new FileHandler("archivo.xml")
file.setLevel(Level.SEVERE)
logger.addHandler(file)
```

como filehandler tiene por defecto el xmlformatter solo instancio el logger con el mensaje que quiero que el handler envie al archivo xml

```
loggerXML.log(Level.SEVERE, "holu")
```

2- Necesito crear una clase que extienda de Handler (WhatsappHandler) la cual se encargará de manejar el envio de mensajes (logs).Ademas necesito sobrescribir el método publis(LogRecord record) ya que necesito invocar al metodo de la clase Whatsapp que envia el mensaje:

```java
public class WhatsAppHandler extends Handler {
    private String telefono;

    public WhatsAppHandler(String tel) {
        this.telefono = tel;
    }

    @Override
    public void publish(LogRecord record) {
        if (record.getLevel().equals(this.getLevel())) {
            Whatsapp.enviar(record.getMessage(), telefono);
        }
    }

    @Override public void flush() {}
    @Override public void close() throws SecurityException {}
}
```

```java
Logger logger = Logger.getLogger("MiLoggerSevere");
logger.setLevel(Level.SEVERE);
wspHandler= new WhatsAppHandler("+541112345678")
wspHandler.setLevel(Level.SEVERE)
logger.addHandler(wspHandler);
```

3-Sí, el programador nota la inversión de control porque no llama directamente a los métodos que hacen el logging.

- El programador solo llama logger.log(...) etc
- No sabe qué Handlers se ejecutarán, en qué orden, ni qué formato tendrán.
- El framework controla qué hacer, cómo, y cuándo hacerlo.A que métodos definidos por el programador que está usando el framework llamar (esto es inversión de control).
- El flujo lo maneja el framework, el programador solo configura o extiende.

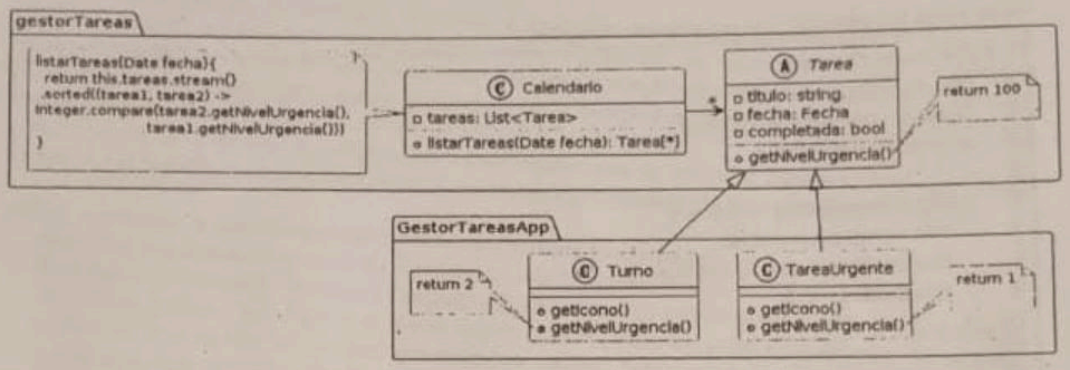
Hay inversión de control porque el flujo de ejecución del logging no lo controla el programador, sino el framework. El programador configura Handlers y Formatters, pero no llama directamente a sus métodos, el framework se encarga.

Parcial 1ra fecha 2024

Ejercicio Frameworks

OO2 - Parcial 8/6/2024

Consideremos una parte de un framework muy simple para la gestión de tareas. El framework define un calendario que permite registrar muchas tareas. Al utilizar este calendario, es necesario especificar qué tareas se van a incluir. Además, ofrece la posibilidad de listar las tareas para una fecha determinada en orden de prioridad, comenzando desde las menos urgentes hasta las más prioritarias.



Para crear una aplicación con el framework, el programador debe:

- Crear una instancia de Calendario y agregarle tareas.
- Las tareas deben ser subclases de Tarea, y puede definirse libremente según se requiera.
- Al definir una tarea tiene que determinar cuál será el nivel de urgencia de la misma. Se espera que las tareas retornen un nivel de urgencia que debe ser un número entero. Cada tarea puede definir un nivel de urgencia para registrar su prioridad, que se utiliza entre otras cosas, para el listado.
- Este framework ordena las tareas para un día del calendario por nivel de urgencia, considerando las de mayor valor como las más urgentes.

GestorTareasApp es una aplicación construida con el framework. En ella los programadores definieron dos tipos de tareas: Turno y TareaUrgente, ambas subclases de Tarea. La tarea urgente tiene un nivel de urgencia 1, mientras que Turno tiene un nivel de urgencia 2.

Tareas:

1) Responda las siguientes preguntas, basándose en el subconjunto de clases y métodos del framework presentado anteriormente:
   a) A los programadores de la aplicación GestorTareasApp, no les gusta el orden ofrecido por nivel de urgencia, considerando las de mayor valor como las más urgentes. ¿Pueden modificar ese orden? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?
   b) Los programadores requieren nuevas tareas para su aplicación. ¿Pueden agregarlas? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?
   c) Los programadores de la aplicación midieron los tiempos de respuestas al momento de ordenar tareas y les parece muy lento. Quieren usar una colección que ordene de manera eficiente. ¿Pueden hacerlo? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?
   d) ¿Se puede implementar una tarea que implemente el nivel de urgencia incrementando su valor al acercarse la fecha límite? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?

2) ¿Observa métodos gancho? ¿Cuáles?
3) Analizando la clase Turno, los programadores de GestorDeTareasApp pueden asegurar si hay una inversión de control? ¿Dónde?

1- a:

No se puede modificar el orden ofrecido por listarTareas, ya que este método se encuentra definido dentro del framework en la clase gestorTareas y no ofrece mecanismos explícitos de extensión ni de sobreescritura para cambiar la política de ordenamiento (que está basada en el método getNivelUrgencia). Por lo tanto, este comportamiento corresponde a un frozen spot.

1-b:

Sí, los programadores pueden agregar nuevas tareas creando nuevas subclases de Tarea. El mecanismo de extensión del framework es la herencia, permitiendo redefinir el método getNivelUrgencia() según la lógica propia de cada tipo de tarea. Por lo tanto, este punto corresponde a un hot spot, ya que permite a los desarrolladores extender el comportamiento del framework.

1-c:

No, los programadores no pueden cambiar la colección utilizada para almacenar las tareas, ya que el framework fija internamente una List<Tarea> en la clase Calendario, sin ofrecer mecanismos de extensión ni reemplazo de esta estructura. Por lo tanto, este aspecto corresponde a un frozen spot.

1-d:

Sí, se puede implementar una tarea que modifique dinámicamente su nivel de urgencia en función de la cercanía a la fecha límite, implementando la lógica deseada en el método getNivelUrgencia() de una subclase de Tarea. Este es un hot spot, ya que el framework permite y espera que los desarrolladores redefinan este método para personalizar el comportamiento.

2:

Sí, se observa un método gancho. El método plantilla listarTareas del framework ordena las tareas utilizando getNivelUrgencia(), el cual es un método gancho que las subclases de Tarea pueden redefinir para controlar el nivel de urgencia.

3:

Sí, hay inversión de control. Los programadores definen subclases de Tarea con sus propios métodos, pero no controlan directamente cuándo ni cómo se invocan estos métodos. Es el framework, a través del método listarTareas, el que controla el flujo de ejecución, invocando getNivelUrgencia() de cada tarea según lo necesite.